Integrantes Caso 3

Samuel Muñoz – 202423981

Jose Luis Velandia – 202416615

1. Introducción

Este informe presenta la versión final del Caso 3, cuyo objetivo fue implementar un sistema concurrente que simula el procesamiento de eventos en un entorno IoT para un campus universitario. El sistema coordina la generación de eventos desde múltiples sensores, detecta comportamientos sospechosos y distribuye los eventos válidos hacia distintos servidores de consolidación y despliegue.

La implementación se desarrolló con primitivas básicas de concurrencia de Java y con dos patrones de espera: espera pasiva y espera semi-activa. Además de cumplir el flujo funcional del caso, el programa valida la consistencia de los conteos al finalizar y verifica que todos los buzones queden vacíos, lo que permite comprobar que la terminación del sistema es correcta.

2. Diseño del sistema

Sensores -> Broker y Analizador -> (Administrador o Clasificadores) -> Servidores de consolidación y despliegue

2.1 Clases principales

- Sensor
  - Extiende Thread.
  - Genera base_eventos * sensorId eventos y los envía al buzón de entrada ilimitado.
  - Cada evento recibe un identificador único y un tipo lógico TYPE_k entre 1 y ns.
- Broker
  - Extiende Thread.
  - Consume eventos del buzón de entrada y decide si son normales o sospechosos.
  - Si el valor aleatorio generado es múltiplo de 8, envía el evento al buzón de alertas; en caso contrario, lo envía al buzón de clasificación.
- Administrator
  - Extiende Thread.
  - Consume alertas del buzón de alertas y determina si la alerta es inofensiva o debe descartarse.
  - Cuando recibe el fin del broker, deposita nc eventos de fin en el buzón de clasificación.
- Classifier
  - Extiende Thread.
  - Consume eventos del buzón de clasificación y los reenvía al servidor correspondiente según su tipo y destino.
  - El último clasificador en terminar envía ns eventos de fin, uno por cada servidor final.
- ServerNode
  - Extiende Thread.
  - Representa cada servidor final de consolidación y despliegue.

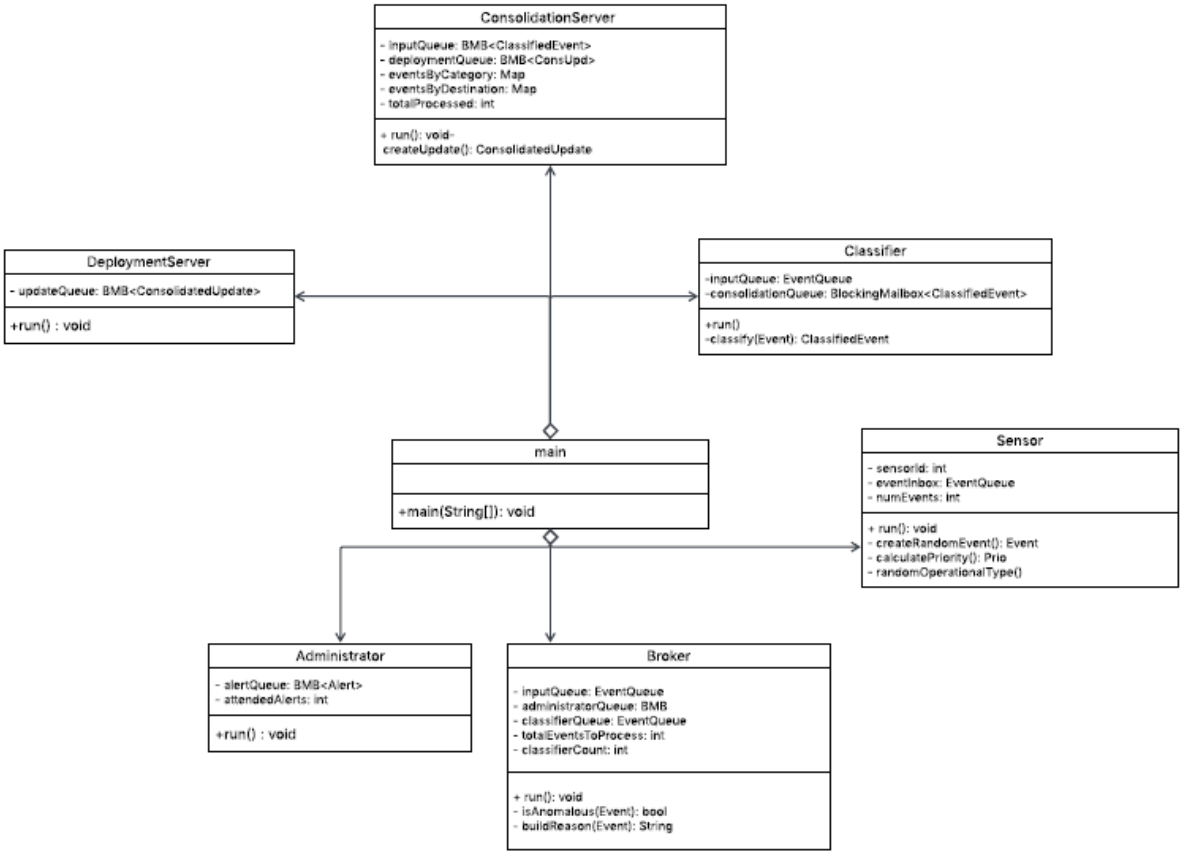
- Procesa eventos de su propio buzón limitado y simula una demora aleatoria entre 100 ms y 1000 ms.
- PassiveMailbox<T>
  - Buzón ilimitado con espera pasiva basada en synchronized, wait() y notifyAll().
- LimitedMailbox<T>
  - Buzón limitado con espera semi-activa mediante reintentos y Thread.yield().
- ClassifierTerminationState
  - Registra cuántos clasificadores han terminado y permite identificar al último en finalizar.
- ConfigReader
  - Lee el archivo de configuración y soporta tanto las claves del enunciado como las claves heredadas de versiones anteriores.

2.2 Flujo

Los sensores generan eventos y los depositan en un buzón de entrada ilimitado. El broker consume esos eventos uno por uno y, para cada uno, genera un valor pseudoaleatorio entre 0 y 200. Si ese valor es múltiplo de 8, el evento se considera sospechoso y se envía al buzón de alertas; de lo contrario, se envía al buzón de clasificación.

El administrador consume las alertas y, por cada una, genera un valor aleatorio entre 0 y 20. Si ese valor es múltiplo de 4, la alerta se considera inofensiva y el evento vuelve al buzón de clasificación; si no, el evento se descarta. Los clasificadores consumen del buzón de clasificación y reenvían cada evento al buzón del servidor que le corresponde. Finalmente, cada servidor procesa sus eventos, simula el tiempo de trabajo y despliega el resultado por consola.

3. Diagrama de Clases



Relaciones con main

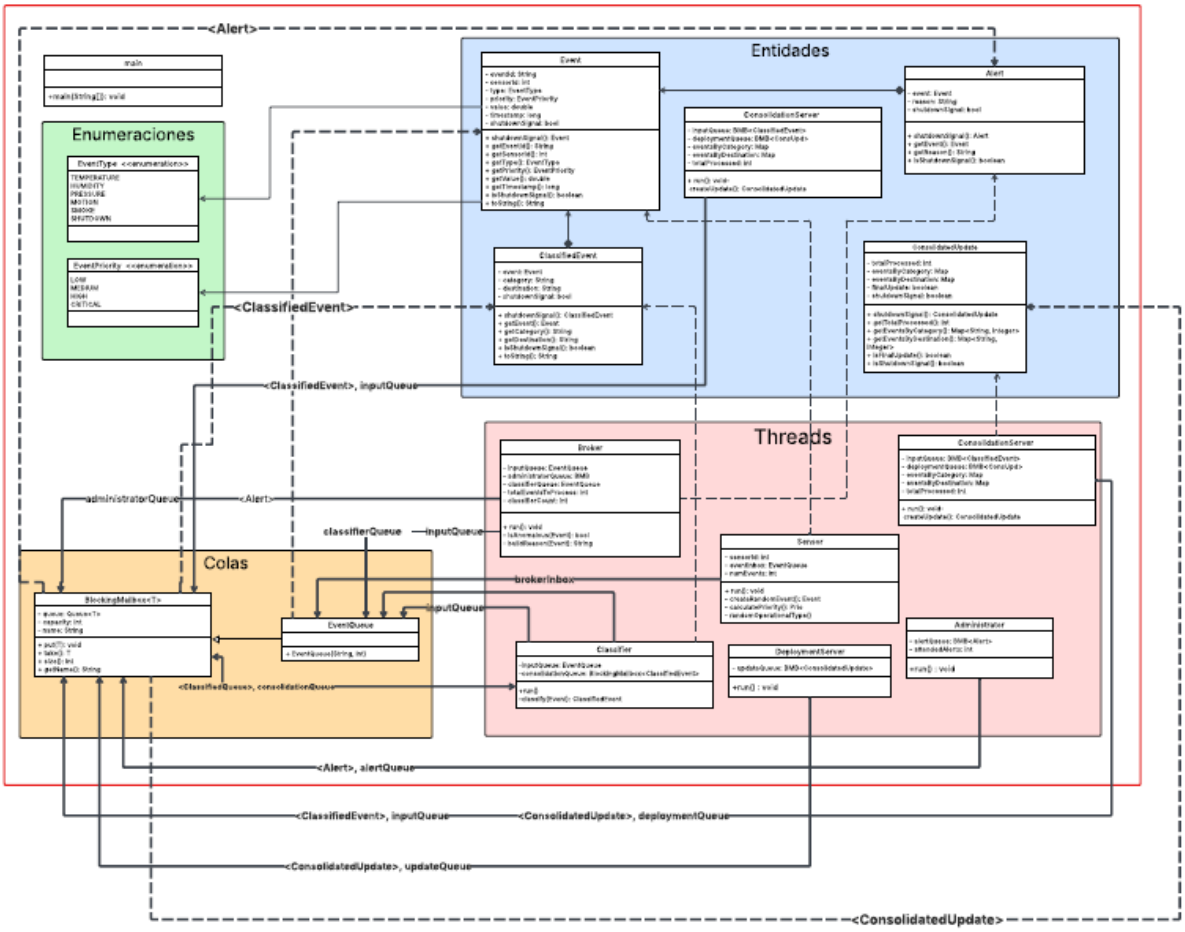


Diagrama completo sin las relaciones con main (ya mostradas anteriormente)

4. Funcionamiento

4.1 Sensores IoT

Cada Sensor es un Thread que genera un número fijo de eventos igual a base_eventos * sensorId. El identificador de cada evento sigue el formato S%02d-E%04d, lo que garantiza unicidad dentro de la ejecución. Además, a cada evento se le asigna un número aleatorio entre 1 y ns, que representa al mismo tiempo el tipo lógico del evento y el servidor final al que debe llegar. En el código este tipo se representa como TYPE_k.

4.2 Broker y Analizador

El broker consume eventos del buzón de entrada y, para cada uno, genera un valor aleatorio entre 0 y 200. Si ese valor es múltiplo de 8, el evento se considera sospechoso y se encapsula en una alerta para ser enviado al administrador. Si no lo es, el evento se reenvía al buzón de clasificación. El broker conoce el número total de eventos esperados y, cuando termina de procesarlos todos, deposita un único evento de fin en el buzón de alertas para notificar al administrador.

4.3 Administrador

El administrador consume las alertas desde el buzón de alertas. Por cada alerta genera un valor aleatorio entre 0 y 20. Si ese valor es múltiplo de 4, la alerta se considera inofensiva y el evento asociado se reenvía al buzón de clasificación. Si no, el evento se descarta. Al recibir el evento de fin del broker, el administrador deposita nc eventos de fin en el buzón de clasificación y luego termina su ejecución.

4.4 Clasificadores

Cada clasificador consume del buzón de clasificación y reenvía cada evento al servidor que le corresponde según su valor TYPE_k. Todos los clasificadores comparten el mismo buzón de clasificación, por lo que cada evento es retirado exactamente una vez. Cuando un clasificador recibe un evento de fin, espera a los demás en una CyclicBarrier. El último clasificador en terminar deposita ns eventos de fin, uno en cada buzón de servidor.

4.5 Servidores de consolidación y despliegue

Cada ServerNode tiene su propio buzón limitado de consolidación y despliegue. El servidor retira un evento, simula su procesamiento con un tiempo aleatorio entre 100 ms y 1000 ms y luego imprime el despliegue asociado al evento procesado. Cuando recibe su evento de fin, termina. De esta forma, cada servidor es responsable de la consolidación y el despliegue de su propia cola.

5. Sincronización

5.1 PassiveMailbox: espera pasiva

La espera pasiva se utiliza en los buzones ilimitados: el buzón de entrada y el buzón de alertas. En este caso, el hilo que intenta consumir cuando la cola está vacía entra en wait() y libera el monitor hasta que otro hilo inserta un elemento y ejecuta notifyAll().

```
public synchronized void put(T item) {
    queue.add(item);
    notifyAll();
}

public synchronized T take() throws InterruptedException {
    while (queue.isEmpty()) {
        wait();
    }
    T item = queue.poll();
    notifyAll();
    return item;
}
```

El uso de while evita problemas por despertares espurios y garantiza que la condición de la cola se vuelva a verificar antes de continuar.

5.2 LimitedMailbox: espera semi-activa

La espera semi-activa se usa en el buzón de clasificación y en los buzones de cada servidor final. En esta estrategia no se usa wait(); en su lugar, el hilo intenta insertar o retirar y, si no puede continuar, reintenta después de ceder el procesador con Thread.yield().

```
public void putSemiActive(T item) throws InterruptedException {
    while (!tryPut(item)) {
        if (Thread.currentThread().isInterrupted()) {
            throw new InterruptedException();
```

```
        }
        Thread.yield();
    }
}

public T takeSemiActive() throws InterruptedException {
    T item = tryTake();
    while (item == null) {
        if (Thread.currentThread().isInterrupted()) {
            throw new InterruptedException();
        }
        Thread.yield();
        item = tryTake();
    }
    return item;
}
```

5.3 Sincronización por pareja

Sensor -> Broker: la sincronización se realiza por medio del PassiveMailbox<Event> del buzón de entrada. Como el buzón es ilimitado, los sensores no bloquean por capacidad; el broker sí puede quedar esperando pasivamente cuando aún no hay eventos para consumir.

Broker -> Administrator: se usa PassiveMailbox<Alert>. El broker deposita alertas y el administrador las consume con espera pasiva. Cuando el broker termina de procesar todos los eventos, deposita un único evento de fin en este mismo buzón.

Broker -> Classifiers y Administrator -> Classifiers: ambos componentes interactúan con el LimitedMailbox<Event> de clasificación. Dado que el buzón es limitado por tam1, tanto el broker como el administrador utilizan espera semi-activa para depositar eventos, y los clasificadores utilizan espera semi-activa para retirarlos.

Classifiers -> ServerNode: cada clasificador deposita en el LimitedMailbox<Event> del servidor correspondiente. Como esos buzones están limitados por tam2, también se utiliza espera semi-activa para la inserción y el retiro.

5.4 Terminación

La terminación ocurre en varias fases. Primero, cada sensor termina al generar todos sus eventos. Luego, el broker termina cuando consume el total esperado y envía un único fin al administrador. Después, el administrador envía nc eventos de fin a los clasificadores. Finalmente, el último clasificador en terminar envía ns eventos de fin, uno a cada servidor final. Main realiza join() de todos los hilos y valida que no queden buzones con elementos.

5.5 Condiciones de carrera y cómo se evitan

Las condiciones de carrera se evitan encapsulando el acceso a las colas compartidas dentro de métodos synchronized. En PassiveMailbox la exclusión mutua protege la cola ilimitada, y en LimitedMailbox los métodos tryPut() y tryTake() serializan el acceso a la cola limitada. Esto garantiza que dos hilos no puedan retirar o insertar simultáneamente sobre la misma estructura.

5.6 Ausencia de deadlock

No se presentan bloqueos mutuos porque el flujo del sistema es unidireccional y cada actor interactúa con un conjunto acotado de buzones. Además, la lógica de terminación está definida

con eventos de fin y una barrera explícita para sincronizar a los clasificadores. De esta forma, todos los hilos tienen una condición clara de salida y el sistema puede terminar sin dejar procesos activos.

6. Validación y Pruebas

6.1 Estrategia de prueba

Para verificar el correcto funcionamiento del sistema se ejecutó el programa con una configuración de prueba ni=3, base_eventos=8, nc=2, ns=3, tam1=10 y tam2=10. Con esta configuración, el total esperado de eventos es 48. Como los valores aleatorios del broker y del administrador cambian en cada ejecución, la validación se centra en invariantes que deben cumplirse en todas las corridas.

Prueba 1: terminación correcta del sistema. Se verificó que todos los hilos terminen correctamente y que Main complete todos los join(). El mensaje final de éxito y la validación de buzones vacíos indican que el sistema terminó sin dejar hilos activos ni elementos pendientes en las colas.

Prueba 2: consistencia del conteo de eventos. Se comprobó que la cantidad total de eventos generados por los sensores coincide con el total esperado, que el broker procesa exactamente ese mismo número y que la suma entre eventos normales y sospechosos vuelve a dar el mismo total.

Prueba 3: consistencia del manejo de alertas. Se validó que el número de alertas recibidas por el administrador coincide con el número de eventos sospechosos reportados por el broker y que, para el administrador, alertas inofensivas + alertas descartadas es igual al total de alertas recibidas.

Prueba 4: consistencia del reenvío a servidores. Se verificó que los clasificadores envían a servidores exactamente la cantidad de eventos esperada, es decir, eventos normales del broker más alertas inofensivas del administrador. Además, se comprobó que la suma de eventos procesados por todos los servidores coincide con la cantidad total enviada por los clasificadores.

Prueba 5: propagación correcta de señales de fin. Se observó que el broker envía un único fin al administrador, que el administrador deposita nc eventos de fin a los clasificadores y que el último clasificador deposita ns eventos de fin a los servidores. Esto permite una terminación ordenada del sistema.

Prueba 6: invariante de buzones vacíos al terminar. Al finalizar la ejecución, Main revisa el tamaño del buzón de entrada, del buzón de alertas, del buzón de clasificación y de cada buzón de servidor. El resultado correcto es que todos tengan tamaño 0.

7. Conclusión

En este trabajo se logró implementar un sistema IoT concurrente que cumple el flujo funcional del caso y que, además, utiliza dos estrategias de espera: espera pasiva en los buzones ilimitados y espera semi-activa en los buzones limitados. La combinación de synchronized, wait(), notifyAll(), Thread.yield(), join() y CyclicBarrier permitió coordinar correctamente los hilos sin recurrir a utilidades avanzadas de concurrencia.

La validación final mostró que los conteos se mantienen consistentes entre sensores, broker, administrador, clasificadores y servidores, y que todos los buzones quedan vacíos al terminar. Esto indica que la terminación del sistema es correcta y que la sincronización adoptada evita condiciones de carrera y bloqueos mutuos en la ejecución normal del programa.